

The Computing of Digital Ecosystems

Gerard Briscoe

Media and Communications

London School of Economics and Political Science

United Kingdom

Philippe De Wilde

Department of Computer Science

Heriot Watt University

United Kingdom

Abstract

A primary motivation for our research in *digital ecosystems* is the desire to exploit the self-organising properties of biological ecosystems. Ecosystems are thought to be robust, scalable architectures that can automatically solve complex, dynamic problems. However, the computing technologies that contribute to these properties have not been made explicit in *digital ecosystems* research. Here, we discuss how different computing technologies can contribute to providing the necessary self-organising features, including Multi-Agent Systems (MASs), Service-Oriented Architectures (SOAs), and distributed evolutionary computing (DEC). The potential for exploiting these properties in *digital ecosystems* is considered, suggesting how several key features of biological ecosystems can be exploited in Digital Ecosystems, and discussing how mimicking these features may assist in developing robust, scalable self-organising architectures. An example architecture, the Digital Ecosystem, is considered in detail. The Digital Ecosystem is then measured experimentally through simulations, considering the self-organised diversity of its evolving agent populations relative to the user request behaviour.

1 Introduction

Digital Ecosystems are distributed adaptive open socio-technical systems, with properties of self-organisation, scalability and sustainability, inspired by natural ecosystems [4, 8, 5], and are emerging as a novel approach to the catalysis of sustainable regional development driven by Small and Medium sized Enterprises (SMEs). Digital Ecosystems aim to help local economic actors become active players in globalisation, valorising their local culture and vocations, and enabling them to interact and create value networks at the global level [14, 43].

We have previously considered the biological inspiration for the technical component of Digital Ecosystems [10, 9], and we will now consider the relevant computing technologies. Based on our understanding of biological ecosystems in the context of Digital Ecosystems [6], we will now introduce fields from the domain of computer science relevant to the creation of Digital Ecosystems. As we are interested in the digital counterparts for the behaviour and constructs of biological ecosystems, instead of simulating or emulating such behaviour or constructs, we will consider what parallels can be drawn. We will start by considering MASs to explore the references to *agents* and *migration* [10, 9]; followed by evolutionary computing and SOAs for the references to *evolution* and *self-organisation* [10, 9].

The value of creating parallels between biological and computer systems varies substantially depending on the behaviours or constructs being compared, and sometimes cannot

be done so convincingly. For example, both have mechanisms to ensure data integrity. In computer systems, that integrity is absolute, data replication which introduces even the most minor change is considered to have failed, and is supported by mechanisms such as the Message-Digest algorithm 5 [39]. While in biological systems, the genetic code is transcribed with a remarkable degree of fidelity; there is, approximately, only one unforced error per one hundred bases copied [32]. There are also elaborate proof-reading and correction systems, which in evolutionary terms are highly conserved [32]. In this example establishing a parallel is infeasible, despite the relative similarity in function, because the operational control mechanisms in biological and computing systems are radically different, as are the aims and purposes. This is a reminder that considerable finesse is required when determining parallels, or when using existing ones.

2 Multi-Agent Systems

A *software agent* is a piece of software that acts, for a user in a relationship of *agency*, autonomously in an environment to meet its designed objectives. A MAS is a system composed of several *software agents*, collectively capable of reaching goals that are difficult to achieve by an individual agent or monolithic system. Conceptually, there is a strong parallel between the software agents of a MAS and the agent-based models of a biological ecosystem [15], despite the lack of evolution and migration in a MAS. There is an even stronger parallel to a variant of MASs, called *mobile agent systems*, in which the mobility also mirrors the migration in biological ecosystems [38].

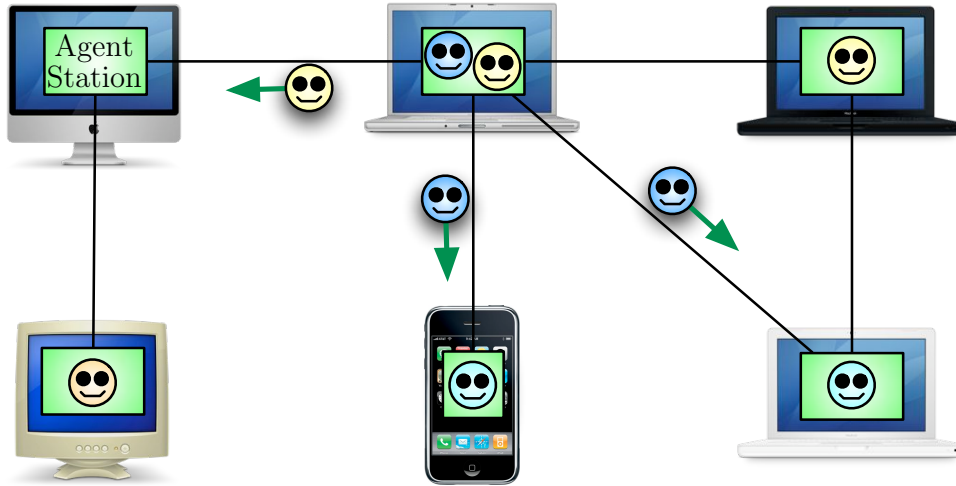


Figure 1: *Mobile Agent System: Visualisation that shows mobile agents as programmes that can migrate from one host to another in a network of heterogeneous computer systems and perform a task specified by its owner.*

The term *mobile agent* contains two separate and distinct concepts: mobility and agency [40]. Hence, mobile agents are software agents capable of movement within a network [38]. The mobile agent paradigm proposes to treat a network as multiple agent-*friendly* environments and the agents as programmatic entities that move from location to location, performing tasks for users. So, on each host they visit mobile agents need software which is responsible for their execution, providing a safe execution environment [38].

Generally, there are three types of design for mobile agent systems [38]: (1) using a specialised operating system, (2) as operating system services or extensions, or (3)

as application software. The third approach builds mobile agent systems as specialised application software that runs on top of an operating system, to provide for the mobile agent functionality, with such software being called an *agent station* [31]. In this last approach, each agent station hides the vendor-specific aspects of its host platform, and offers standardised services to visiting agents. Services include access to local resources and applications; for example, web servers or *web services*, the local exchange of information between agents via message passing, basic security services, and the creation of new agents [31]. Also, the third approach is the most platform-agnostic, and is visualised in Figure 1.

3 Service-Oriented Architectures

To evolve high-level software components in Digital Ecosystems, we propose taking advantage of the *native* method of software advancement, human developers, and the use of *evolutionary computing* for *combinatorial optimisation* of the available software services. This involves treating developer-produced software services as the functional building blocks, as the base unit in a genetic-algorithms-based process. Our approach to evolving high-level software applications requires a modular reusable paradigm to software development, such as SOAs. SOAs are the current state-of-the-art approach, being the current iteration of interface/component-based design from the 1990s, which was itself an iteration of event-oriented design from the 1980s, and before then modular programming from the 1970s [22]. Service-oriented computing promotes assembling application components into a loosely coupled network of services, to create flexible, dynamic business processes and agile applications that span organisations and computing platforms [37]. This is achieved through a SOA, an architectural style that guides all aspects of creating and using business processes throughout their life-cycle, packaged as services. This includes defining and provisioning infrastructure that allows different applications to exchange data and participate in business processes, loosely coupled from the operating systems and programming languages underlying the applications. Hence, a SOA represents a model in which functionality is decomposed into distinct units (services), which can be distributed over a network, and can be combined and reused to create business applications [37].

A SOA depends upon service-orientation as its fundamental design principle. In a SOA environment, independent services can be accessed without knowledge of their underlying platform implementation. Services reflect a *service-oriented* approach to programming that is based on composing applications by discovering and invoking network-available services to accomplish some task. This approach is independent of specific programming languages or operating systems, because the services communicate with each other by passing data from one service to another, or by co-ordinating an activity between two or more services [37]. So, the concepts of SOAs are often seen as built upon, and the development of, the concepts of modular programming and distributed computing [22].

SOAs allow for an information system architecture that enables the creation of applications that are built by combining loosely coupled and interoperable services. They typically implement functionality most people would recognise as a service, such as filling out an online application for an account, or viewing an online bank statement [22]. Services are intrinsically unassociated units of functionality, without calls to each other embedded in them. Instead of services embedding calls to each other in their source code, protocols are defined which describe how services can talk to each other, in a process known as orchestration, to meet new or existing business system requirements. This is allowing an increasing number of third-party software companies to offer software services, such that SOA systems will come